

BC7215(A)
Driver Library V4.x
User Manual

Index

Introduction.....	3
Usage.....	3
Configuration File.....	3
Data Types.....	5
Hardware Interface Functions to Be Provided by the User.....	6
I/O Port Operation Functions.....	6
UART Operation Functions.....	7
User API Functions.....	8
1. UART Data Processing Function.....	8
2. Status Control Functions.....	8
3. Query Functions.....	9
4. Receive-Related Functions.....	10
5. Transmit-Related Functions.....	11
6. Utility Functions.....	12
7. Hardware Interface Configuration Functions.....	14

Introduction

This BC7215(A) C driver library provides an interface to the BC7215(A). When using the BC7215(A), users can call the functions in this driver library to perform various operations for the BC7215(A) chip.

This driver library includes the operations required for using the BC7215(A), but it does not include hardware-specific operations. All hardware interfaces must be provided by the user as functions.

Usage

This library is provided in the form of C source files. It contains the following four files:

`bc7215_lib_config.h`

`bc7215_types.h`

`bc7215_lib.h`

`bc7215_lib.c`

The user only needs to add the following line to a C source file:

```
#include "bc7215_lib.h"
```

Then the functions provided by this library can be used in the program.

This driver library requires a compiler that supports at least C89 or a later version of the C language.

Configuration File

This driver library includes a configuration file named `bc7215_lib_config.h`, which defines some parameters related to the driver library. Users may adjust them as needed before use. The main configuration parameters are as follows.

`TX_HW_FLOW_CONTROL`

This setting is mainly related to sending data to the BC7215(A) chip; that is, whether the system's UART transmission uses hardware flow control. If the hardware system supports serial-port flow control, such as on a PC or on many MCUs such as the STM32 series, it is recommended to use hardware flow control to control data transmission to the BC7215(A). In this case, the BUSY signal of the BC7215 should be used as the permission signal for serial transmission, which is usually the CTS signal on a PC. Using hardware flow control can simplify software design and reduce system overhead. Defining this value as `1` indicates that hardware flow control is used. The default is `0`.

The hardware flow control mentioned here is from the point of view of this driver library. In other words, the BC7215(A) driver library does not need to handle flow control itself; it is completed by a lower-level system. If the user implements software-based data transmission controlled by the BUSY signal and can guarantee that BUSY is low whenever each byte is sent onto the serial data line, then from the point of view of the driver library this is also equivalent to having hardware flow control, and this option may be selected.

When hardware flow control is not used, the user must provide a function for reading the state of the BC7215(A) BUSY pin.

USE_UART_INTERRUPT

This setting is mainly related to receiving data from the BC7215(A); that is, whether interrupts are used to process serial-port data. In MCU systems, interrupts are commonly used to process received serial data. In more complex systems with an operating system, such as Windows and Linux, including Arduino systems, the system provides a low-level serial-port driver. The received data is automatically written into a serial buffer, and the user program only needs to read the serial buffer instead of directly operating the UART hardware. Defining this value as `1` indicates that UART data is received by interrupt. The default is `1`.

When this value is defined as `1`, the user should call `bc7215_process_uart_data()` in the UART receive interrupt, and the user must provide two functions to enable and disable UART interrupts. The driver library disables UART interrupts during certain operations to prevent conflicts.

If the system being used can automatically store received UART data into a buffer, including a function implemented by the user in software, and the UART data user only needs to read the UART buffer periodically or in response to some event, this option should be set to `0`.

PROCESS_IN_DIFFERENT_THREAD

This setting mainly applies to the use of a real-time multitasking operating system. It indicates whether UART data processing, that is, calling `bc7215_process_uart_data()`, is performed in a different thread. In systems that support multithreading, even when interrupts are not used, UART data processing and the main BC7215(A) operations may still be placed in different threads, which is not recommended. In this case, similar to an interrupt-based system, the user must provide two mutex operation functions. During driver-library initialization, these functions are passed to the driver library through `bc7215_config_mutex_lock_func()` and `bc7215_config_mutex_unlock_func()`. The driver library calls these two functions during related operations to ensure that no resource-access conflict occurs. When this value is `1`, it means that the call to `bc7215_process_uart_data()` and other BC7215 operations are in different threads. The default is `0`.

ENABLE_RECEIVING

This parameter controls whether the receive decoding function of the driver library is enabled, including receiving raw data packets and receiving format information packets. The default value of this parameter is `1`, meaning enabled. If the user does not need the BC7215 receive decoding function, this value may be changed to `0`. After the receive function is disabled, the related functions will also become unavailable. Because the receive decoding function occupies most of the

required memory and program space, disabling it will greatly reduce the size of this driver library and the memory it occupies.

ENABLE_FORMAT

Under the receive function, there is another further control parameter that controls whether receiving format information is enabled. The default value is `1`, meaning enabled. If it is not needed, it may be changed to `0`. In some applications, such as data communication, it is not necessary to obtain the format information of the infrared signal. In that case, this function can be disabled. Because the format information packet is 33 bytes, this saves 33 bytes of RAM and part of the program space.

ENABLE_TRANSMITTING

This parameter controls whether the infrared transmit-related functions are enabled. The default value is `1`, meaning enabled. It may be changed to `0` to disable these functions. Disabling the transmit-related functions can save part of the program space.

BC7215_MAX_RX_DATA_SIZE

This defines the maximum length of a receivable raw data packet in bytes. The range is 1 to 512. For actual data sent by remote controls, audio/video equipment usually uses data lengths within 8 bytes, while air conditioners generally use data lengths within 48 bytes. The larger this defined value is, the more memory this driver library will occupy.

BC7215_CRC8_POLY

This function library provides a CRC-8 calculation utility function. This parameter defines the polynomial value used for CRC-8 calculation. The default value is `0x07`, and users may modify it as needed.

Data Types

This driver library defines two types of data packets.

Raw Data Packet

The memory size occupied by a raw data packet is determined by a user-defined constant in the configuration file:

```
typedef struct bc7215DataMaxPkt_t
{
    uint16_t bitLen;
    uint8_t data[BC7215_MAX_RX_DATA_SIZE];
} bc7215DataMaxPkt_t;
```

This type defines the largest data packet that can be processed.

```
typedef struct bc7215DataVarPkt_t
{
    uint16_t bitLen;
    uint8_t data[1];
} bc7215DataVarPkt_t;
```

This type defines the structure of a data packet and is used to process variable-length data packets, mainly through pointers.

Format Data Packet

The format data packet has a fixed size and occupies 33 bytes:

```
typedef struct {
    union {
        struct {
            uint8_t sig : 6;
            uint8_t c56k : 1;
            uint8_t noCA : 1;
        } bits;
        uint8_t inByte;
    } signature;
    uint8_t format[32];
} bc7215FormatPkt_t;
```

In addition, the driver library defines three function-pointer types to represent the signatures of the hardware interface functions that must be provided by the user. These are, respectively: no input parameters and no return value; one input parameter of 8-bit width and no return value; and no input parameters with an 8-bit return value:

```
typedef void (*voidFunction_t) (void);
typedef void (*uint8ParamFunction_t) (uint8_t);
typedef uint8_t (*readIOFunction_t) (void);
```

Hardware Interface Functions to Be Provided by the User

The hardware-related functions that the user must provide include the following two categories. For details, see the section "Hardware Interface Configuration Functions".

I/O Port Operation Functions

These functions include I/O port writing and state reading. Such operations usually require only one instruction to complete, but the user must wrap them as functions so that the driver library can call them. They include the following.

MOD pin set-high function:

This function sets the MOD pin of the BC7215(A) to a high level. If the MOD pin is grounded or pulled high by hardware, this function does not need to be provided. In that case, however, note that ``bc7215_set_rx()`` and ``bc7215_set_tx()`` must not be called; otherwise the program will crash.

MOD pin set-low function:

This function sets the MOD pin of the BC7215(A) to a low level. If the MOD pin is grounded or pulled high by hardware, this function does not need to be provided. In that case, however, note that ``bc7215_set_rx()`` and ``bc7215_set_tx()`` must not be called; otherwise the program will crash.

MOD pin read function:

This function reads the current state of the MOD pin. This function must be provided. If the MOD pin is fixed high or low by hardware, the function may simply return ``0`` or ``1``; it does not need to read the MOD pin.

BUSY pin read function:

This function reads the current state of the BUSY pin. When the configuration is set not to use hardware flow control, this function must be provided. In this case, all operations involving sending data to the BC7215(A), including the function ``bc7215_set_rx_mode()`` for changing the operating mode in receive mode, require this function. If the BUSY pin is not connected, this function should be set to return ``0`` directly.

UART Operation Functions

When the BC7215(A) is in receive mode, it actively sends the received data and infrared format information through the UART. Therefore, the host side must be able to receive UART input data at any time. The functions that the user must provide include the following.

The following three functions must be provided only when the configuration is set to "use UART interrupt".

UART interrupt enable function:

This function enables the UART interrupt. If the hardware UART being used includes multiple different interrupts, this specifically refers to the UART receive interrupt. If the infrared receive decoding function is not used, this function does not need to be provided.

UART interrupt disable function:

This function disables the UART interrupt. If the infrared receive decoding function is not used, this function does not need to be provided.

UART receive interrupt handler function:

This function does not need to be provided to this driver library, but the user must call the driver library's UART data processing function inside the interrupt handler.

The following function must be provided.

UART single-byte send function:

The user must provide a serial-port send function whose function is to send one byte to the BC7215(A) through UART. The requirements for this function differ depending on the system:

- - In a system where UART transmission does not use hardware flow control, this function must not return until the byte has been completely sent from the UART.
- - In a system that uses hardware flow control, this function only needs to write the data into the transmit buffer and may then return. It does not need to wait until the data is actually transmitted.

All operations involving sending data to the BC7215(A), including the function ``bc7215_set_rx_mode()`` for changing the operating mode in receive mode, require this function.

User API Functions

The user API functions are the functions provided by the driver library to the user. The user can call them directly in the program to operate the BC7215(A) and perform various functions. They include the following seven categories:

1. - UART data processing function
2. - Status control functions
3. - Query functions
4. - Receive-related functions
5. - Transmit-related functions
6. - Utility functions
7. - Hardware interface configuration functions

1. UART Data Processing Function

```
void bc7215_process_uart_data(uint8_t data);
```

In a system that uses UART interrupts, the user must call this function in the UART receive interrupt. The input parameter is each received UART data byte.

In a system where the low-level UART operation is managed by the system and the user does not directly access UART hardware, the user must call this function one byte at a time when reading the UART input buffer, using each received byte as the parameter, until the receive buffer is empty.

2. Status Control Functions

The user controls the operating state of the BC7215 through this category of functions.

```
void bc7215_set_rx(void);
```

Sets the operating mode of the BC7215(A) to receive, that is, infrared decoding mode. If the BC7215(A) is transmitting infrared data when this function is called, after this function is called the BC7215(A) may need to finish transmitting the current infrared bit before switching to receive state. This may take up to 20 ms. The user should ensure that no other operation is performed on the BC7215(A) during this period.

```
void bc7215_set_tx(void);
```

Sets the operating mode of the BC7215(A) to transmit, that is, infrared encoding mode. After this function is called, the BC7215(A) chip may need up to 70 ms to complete the mode switching. The user should ensure that no other operation is performed on the BC7215(A) during this period.

```
void bc7215_set_shutdown(void);
```

In transmit mode, sets the BC7215(A) to shutdown mode. After this function is called, the `F7 00` command is sent to the BC7215(A).

This function takes effect only in transmit mode. If it is called in receive mode, because the last data sent is `0x00`, the receive mode of the BC7215(A) will become simple mode. For details, refer to the BC7215(A) data sheet. After this function is called, the user may query the command execution status.


```
void bc7215_set_rx_mode(uint8_t mode);
```

In receive mode, sets the receive decoding mode. For details, refer to the BC7215(A) data sheet. The lowest two bits of `mode` determine the operating mode after the command is executed.

3. Query Functions

The user queries the operating state of the BC7215(A) through this category of functions.

```
uint8_t bc7215_data_ready(void);
```

Queries whether a valid infrared data packet has been received from the BC7215(A). When valid data is available, it returns `1`; otherwise it returns `0`. This query is valid only in receive mode. If this function is called in transmit mode, it will always return `0`. If the receive function is disabled in the configuration, this function is unavailable.

The valid-data state of infrared data is cleared in the following three cases:

1. New data is received through the UART.
2. The function `bc7215\_get\_data()` for obtaining a data packet is called.
3. The function `bc7215\_clr\_data()` for clearing a data packet is called.

```
uint8_t bc7215_format_ready(void);
```

Queries whether format data has been received from the BC7215(A). In complex mode, in addition to outputting raw data, the BC7215(A) also outputs the format information of the received infrared signal. Generally speaking, if format information is received, raw data must also have been received before it. However, there is one case in which the total length of the raw data plus the format information exceeds the length of the driver library buffer. In that case, the previously received raw data will be overwritten, and only the format information is available. To obtain the raw data in this situation, the BC7215(A) must be set to simple mode and the infrared signal must be received again.

When valid format data is available, this function returns `1`; otherwise it returns `0`. This query is valid only in receive mode. If this function is called in transmit mode, it will always return `0`. If the receive function is disabled in the configuration, this function is unavailable.

The valid-format-data state is cleared in the following three cases:

1. New data is received through the UART.
2. The function `bc7215\_get\_format()` for obtaining a format packet is called.
3. The function `bc7215\_clr\_format()` for clearing a data packet is called.

```
uint8_t bc7215_cmd_completed(void);
```

In transmit state, queries whether a command has completed execution. The commands that can be queried are transmit commands and shutdown commands.

The BC7215(A) chip has a 16-byte internal receive buffer. Although the infrared transmission rate is relatively low, when the data amount is within 16 bytes, the transmit function returns almost immediately. However, the actual process of sending the data through infrared is completed by the BC7215(A) chip and requires a relatively long time. Sometimes the user needs to know when transmission is complete, for example, when

ensuring the time interval between two consecutive transmissions. In that case, this function can be used to query the completion time of the previous transmission.

The shutdown command executes immediately. The user can use this function to query whether the BC7215(A) has entered shutdown state.

A return value of `1` indicates that the command execution is complete, and a return value of `0` indicates that it has not yet completed. If this function is executed in receive state, it will always return `1`.

```
uint8_t bc7215_is_busy(void);
```

This function was added in version V4.0. In transmit mode, its function is equivalent to ``bc7215_cmd_completed()``, but the returned result uses inverted logic. In receive mode, the return value of this function indicates whether the BC7215(A) chip is in the process of receiving a data packet or format packet; that is, data has already been received but the end marker ``0x7A`` has not yet been received. A return value of `0` indicates that the chip is idle, and a return value of `1` indicates that the chip is busy.

4. Receive-Related Functions

```
uint16_t bc7215_get_len(void);
```

Obtains the data length of the received raw data packet. This length is the actual number of bits in the data, namely the ``bitLen`` field in ``bc7215DataVarPkt_t``. The actual data length copied by the ``bc7215_get_data()`` function is the number of bytes corresponding to this value plus 2. For example, if ``bitLen`` here is 9, the raw data requires 2 bytes, and the actual number of bytes copied will be 4.

If the raw data packet is unavailable, the returned result will be `0`.

The user can use this function to check the data length before obtaining the raw data, in order to prevent memory overflow or to dynamically allocate memory.

If the receive function is disabled in the configuration, this function is unavailable.

```
uint16_t bc7215_dpkt_size(void);
```

Obtains the size of the received data packet in bytes. This function is equivalent to the function above that obtains the bit length, but it returns the byte count of the entire data packet, saving the user from manually performing the conversion. Because the received data packet is determined by the infrared transmitter, data exceeding expectations may be received. The user can check the data packet size before obtaining the packet to prevent memory overflow.

If the receive function is disabled in the configuration, this function is unavailable.

```
uint8_t bc7215_get_data(bc7215DataVarPkt_t* target);
```

Reads the raw data packet. The input parameter is a pointer of type ``bc7215DataVarPkt_t``. After the command is executed, the received raw data packet is copied to the destination address pointed to by this pointer. The return value is the status signature byte of the raw data packet, used to quickly determine the state of the received data. If the raw data in the buffer is no longer available, for example because it has been overwritten by new data, ``0xff`` is returned.

After this function executes, the state of ``bc7215_data_ready()`` is cleared.

The user must ensure that the memory space pointed to by the target ``target`` is large enough to accommodate the received raw data. If the received data length is greater than the memory space pointed to by ``target``, executing this function will cause memory overflow and unpredictable consequences. The user should first use ``bc7215_get_len()`` to obtain the data length, allocate memory or check the size of the memory space, and then call this function.

If the receive function is disabled in the configuration, this function is unavailable.

```
uint8_t bc7215_get_raw(void* addr, uint8_t size);  
uint16_t bc7215_get_raw(void* addr, uint16_t size);
```

Obtains the raw data in the received data packet. This function is similar to the packet-reading function above, but it reads only the raw data and does not include bit-length information. It can also output to any address, making it more suitable for use in infrared communication. Depending on the ``BC7215_MAX_RX_DATA_SIZE`` parameter in the configuration file, when this value is less than 256, the input ``size`` and the return value are both of the 8-bit ``uint8_t`` type. When this value is greater than or equal to 256, these two parameters become the ``uint16_t`` type.

``size`` is the number of bytes expected to be read. This value does not have to be the same as the actual number of bytes received. It may be smaller or larger than the number of bytes actually received. When it is smaller, the number of bytes returned follows the value of ``size``; when ``size`` is larger than the number of bytes actually received, only the actual number of bytes received is read. The return value of the function is the actual number of bytes read.

If the receive function is disabled in the configuration, this function is unavailable.

```
uint8_t bc7215_get_format(bc7215FormatPkt_t* target);
```

Reads the format data packet. The input parameter is a pointer of type ``bc7215FormatPkt_t``. After the command is executed, the received format data packet is copied to the destination address pointed to by this pointer. The return value is the signature byte of the format data packet. If the raw data in the buffer is no longer available, for example because it has been overwritten by new data, ``0xff`` is returned.

After this function executes, the state of ``bc7215_format_ready()`` is cleared.

This function always copies 33 bytes of data to the destination address. The user must ensure that the destination address has enough space to hold the format data packet.

If the receive function is disabled in the configuration, this function is unavailable.

5. Transmit-Related Functions

```
void bc7215_load_format(const bc7215FormatPkt_t* source);
```

Loads format data into the BC7215(A) chip. After this function is called, the format information data pointed to by the pointer ``source`` is loaded into the BC7215(A) chip.

If the transmit function is disabled in the configuration, this function is unavailable.

```
void bc7215_IR_tx(const bc7215DataVarPkt_t* source);
```

Transmits infrared data. Calling this function causes the BC7215(A) to send the raw data packet pointed to by ``source`` through infrared.

The format used for transmission is the last loaded format. If the device has switched from receive mode to transmit mode and no format data has been loaded, the format of the last received infrared signal will be used. If the data to be sent is less than 16 bytes, or 128 bits, this function writes the data into the internal buffer of the BC7215(A) and then returns immediately. If the data is more than 16 bytes, it returns after the last 16 bytes have been written into the buffer.

The specific time required for infrared transmission is determined by the infrared modulation format used. Usually each byte requires several milliseconds. The user can query whether infrared transmission is complete through the ``bc7215_is_busy()`` function. If the transmit function is disabled in the configuration, this function is unavailable.

```
void bc7215_send_raw(const void* rawData, uint16_t size);
```

Sends raw data. The inputs are a pointer to the raw data and the data length. The data pointed to by the pointer does not need to be in the BC7215(A) data-packet format. This function is convenient for data communication, but it can send only complete bytes. The maximum data size is 512 bytes. However, because the infrared transmission rate is not high, sending a long data packet will take a relatively long time and is easily affected by external interference. It is recommended to keep each transmission within a range of several dozen bytes.

```
void bc7215_set_C56K(bc7215FormatPkt_t* target);
```

Sets the control bit in the signature byte of the format data packet. After setting, the ``C56K`` bit is set. After the data packet is loaded, the BC7215(A) will transmit using a 56 kHz carrier.

```
void bc7215_clr_C56K(bc7215FormatPkt_t* target);
```

Clears the control bit in the signature byte of the format data packet. After setting, the ``C56K`` bit is cleared, restoring the default state. After the data packet is loaded, the BC7215 will transmit using a 38 kHz carrier.

```
void bc7215_set_NOCA(bc7215FormatPkt_t* target);
```

Sets the control bit in the signature byte of the format data packet. After setting, the ``NOCA`` bit is set. After the data packet is loaded, the BC7215(A) output will not use a carrier and will be pure high/low level output.

```
void bc7215_clr_NOCA(bc7215FormatPkt_t* target);
```

Sets the control bit in the signature byte of the format data packet. After setting, the ``NOCA`` bit is cleared, restoring the default state. After the data packet is loaded, the BC7215(A) will output using a 38 kHz or 56 kHz carrier.

6. Utility Functions

The driver library also provides some utility functions. These functions do not directly operate the BC7215(A) chip, but are operations frequently needed by users when using the BC7215(A).

When the maximum ``BC7215_MAX_RX_DATA_SIZE`` is less than 256 bytes:

```
uint8_t bc7215_crc8(uint8_t* data, uint8_t len);
```

Or when the maximum data packet is greater than or equal to 256 bytes:

```
uint8_t bc7215_crc8(uint8_t* data, uint16_t len);
```

CRC-8 calculation function. If the BC7215 is used for data communication, it may be necessary to add CRC checking to data packets to improve reliability. Because the data packets suitable for use with the BC7215 are generally small, with a recommended data length within 16 bytes, and because the infrared communication rate is low, using an 8-bit CRC check better meets the requirements: it helps prevent errors without excessively increasing communication time.

The function has two input parameters. The first is a pointer to the data, and the second is the length of the data for which the CRC is to be calculated. The length here refers to the byte length, not the bit length in the data packet. This function can calculate CRC only by byte. The return value is the calculated CRC result. The polynomial used for CRC calculation is `0x07` by default in this driver library and is defined in the `bc7215\_lib\_config.h` file. Users may modify it as needed.

```
uint16_t bc7215_cal_size(const bc7215DataVarPkt_t* dataPkt);
```

Calculates the data packet size. This function returns the size, in bytes, of the data packet in the memory pointed to by `dataPkt`, including the raw data part and the bit-length part.

```
void bc7215_copy_dpkt(bc7215DataVarPkt_t* target, const  
bc7215DataVarPkt_t* source);
```

Data packet copy function. This function copies the data packet pointed to by `source` to the address of `target`. It is worth noting that this function supports overlap between the source and target data packets. In that case, it actually performs a data-packet move. For example, the address of `target` may differ from `source` by only 1 byte, which is equivalent to moving the `source` data packet forward or backward in memory by one byte.

```
uint8_t bc7215_compare_dpkt(uint8_t sig, const bc7215DataVarPkt_t*  
pkt1, const bc7215DataVarPkt_t* pkt2);
```

Compares two data packets. This function compares whether two data packets are the same. If they are the same, the function returns `1`; if they are different, it returns `0`.

The end of infrared data may not be a complete byte. This function supports comparison of incomplete bytes. Because different encoding formats may arrange data with either MSB first or LSB first, signature-byte information is also required to determine the data arrangement direction. The function assumes that the signature bytes of the two data packets are the same. If the two data packets use different modulation methods, their data lengths and data are very likely to be different as well.

7. Hardware Interface Configuration Functions

This category of functions informs the driver library of the hardware interface functions provided by the user, so that the driver library can perform the required hardware operations. These functions must be called before all other library functions are used.

```
void bc7215_config_uart_int_en_func(voidFunction_t);
```

Configures the UART interrupt enable function. When UART interrupts are used, the user must provide a function that enables the UART interrupt. Different controllers may have different interrupt designs. If the UART has multiple interrupts available, the interrupt referred to here is the UART receive interrupt.

The input parameter of ``bc7215_config_uart_int_en_func()`` is a pointer to this interrupt enable function. The interrupt enable function must have the following signature: no input parameters and no return value:

```
void function_name(void);
```

If the receive function is disabled in the configuration, or if the configuration is set not to use UART interrupts, this function is unavailable.

```
void bc7215_config_uart_int_dis_func(voidFunction_t);
```

Configures the UART interrupt disable function. When UART interrupts are used, the user must provide a function that disables the UART interrupt. Different controllers may have different interrupt designs. If the UART has multiple interrupts available, the interrupt referred to here is the UART receive interrupt.

The input parameter of ``bc7215_config_uart_int_dis_func()`` is a pointer to this interrupt disable function. The interrupt disable function must have the following signature: no input parameters and no return value:

```
void function_name(void);
```

If the receive function is disabled in the configuration, or if the configuration is set not to use UART interrupts, this function is unavailable.

```
void bc7215_config_mutex_lock_func(voidFunction_t);
```

Configures the mutex lock function. When a real-time multitasking operating system is used and the call to ``bc7215_process_uart_data()`` is not in the same thread as other driver-library functions, the user needs to provide a mutex mechanism to prevent resource-access conflicts.

```
void bc7215_config_mutex_unlock_func(voidFunction_t);
```

Configures the mutex unlock function. This is the unlock function corresponding to the lock function above.

```
void bc7215_config_uart_send_byte(uint8ParamFunction_t);
```

Configures the UART single-byte send function. The user must provide a function that sends a single byte through UART. When hardware flow control is not used, this function may return only after the byte has been completely sent from the UART. When hardware flow control is used, the function only needs to write the data into the transmit buffer and may then return.

The input parameter of ``bc7215_config_uart_send_byte()`` is a function pointer pointing to this send function.

The single-byte send function must have the following signature: the input parameter is a ``uint8_t`` data value and there is no return value.

```
void function_name(uint8_t);
```

```
void bc7215_config_mod_set_high_funct(voidFunction_t);
```

配置MOD引脚置高电平函数。用户必须提供一个函数，该函数的功能为将BC7215(A)的MOD置为高电平。**bc7215_config_mod_set_high_funct()**函数的输入参数为指向此MOD置高函数的指针。MOD置高电平函数必须具有如下的签名特征：无输入参数，无返回值。

```
void function_name(void);
```

```
void bc7215_config_mod_set_low_funct(voidFunction_t);
```

Configures the MOD pin set-low function. The user must provide a function whose function is to set the MOD pin of the BC7215(A) to a low level. The input parameter of ``bc7215_config_mod_set_low_funct()`` is a pointer to this MOD set-low function. The MOD set-low function must have the following signature: no input parameters and no return value.

```
void function_name(void);
```

```
void bc7215_config_read_mod_funct(readIOFunction_t);
```

Configures the MOD state read function. The user must provide a function whose function is to read the state of the MOD pin of the BC7215(A). The input parameter of ``bc7215_config_read_mod_funct()`` is a pointer to this function. The MOD read function must have the following signature: no input parameters and returns a value of type ``uint8_t``. When MOD is low, it returns ``0``; when MOD is high, it returns a nonzero value:

```
uint8_t function_name(void);
```

```
void bc7215_config_read_busy_funct(readIOFunction_t);
```

Configures the BUSY state read function. When UART hardware flow control is not used, the user must provide a function whose function is to read the state of the BUSY pin of the BC7215(A). The input parameter of ``bc7215_config_read_busy_funct()`` is a pointer to this function. The BUSY read function must have the following signature: no input parameters and returns a value of type ``uint8_t``. When BUSY is low, it returns ``0``; when BUSY is high, it returns a nonzero value:

```
uint8_t function_name(void);
```